

ReactJS Interview

Most Asked Questions

By - Dev Aditya | Roshan Kr

It is a comprehensive guide of the most frequently asked questions in ReactJS Job Interviews. It includes a set of carefully selected questions, frequently asked by MNCs companies while interviewing potential candidates.

1.  **Features of ReactJs**
2.  **Life-cycle**
3.  **Class Component**
4.  **Redux**
5.  **Virtual DOM**
6.  **Real vs Virtual DOM**
7.  **What is JSX and how browser understands it?**
8.  **ES5 vs ES6 of React Syntax**
9.  **Error Handling in React**
10.  **How To Call Child Function From Parent Component In React**
11.  **Fragment in React**
12.  **Stateful & Stateless Component**
13.  **HOC (higher-order component)**
14.  **Context API**
15.  **useEffect Hook as a Lifecycle Hook**
16.  **Controlled and uncontrolled components**
17.  **UseMemo Vs UseCallback**
18.  **Code Splitting**
19.  **React is One way or two way Data Binding ?**
20.  **Pure component**
21.  **What is the difference between state and props?**
22.  **Coding 1 - Stopwatch Timer**
23.  **Coding 2 - Simple Pagination**
24.  **Coding 3 - Fetch Api (Custom Hook)**
25.  **Live Interview - MNC companies Interview videos**
26.  **Projects**
27.  **Important Links**

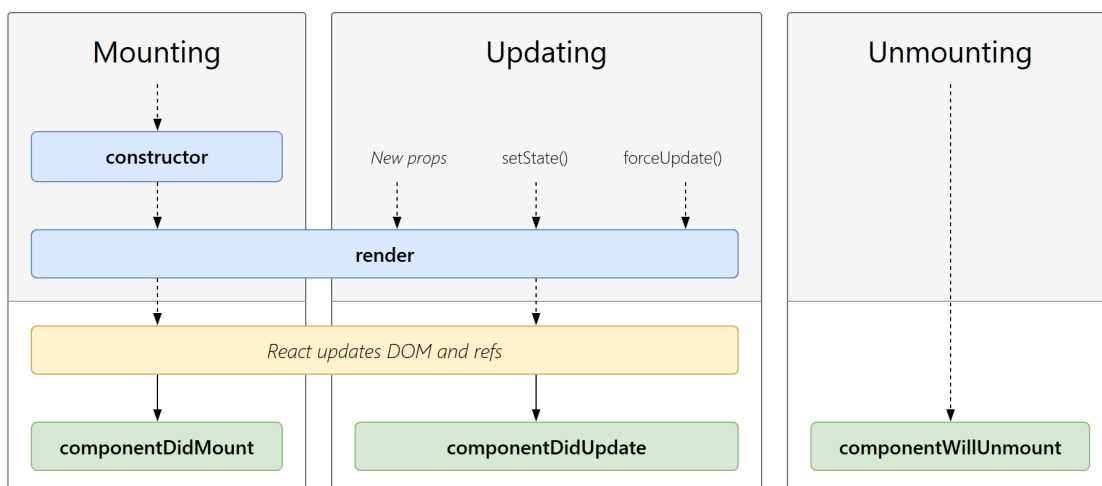
✓ Features of ReactJs

1. **Reusable Components:** React allows developers to build reusable UI components that can be easily composed to create complex user interfaces.
2. **Virtual DOM:** React uses a virtual DOM, which improves application performance by limiting the number of changes that need to be made to the actual DOM.
3. **Unidirectional Data Flow:** React uses a unidirectional data flow model, which makes it easier to understand and manage the state of an application.
4. **Strong Community:** React has a strong and active community of developers, which means that there are many resources available to help developers learn and use the library, and many third-party libraries and tools that are built to work with React.
5. **Popularity:** React is widely used by many companies and has a large user base, this makes it popular among developers and easy to find and hire developers with React skills.
6. **Flexibility:** React can be used with a variety of different programming languages and can be integrated into different types of projects, making it a versatile choice for many different use cases.

✓ Life-cycle

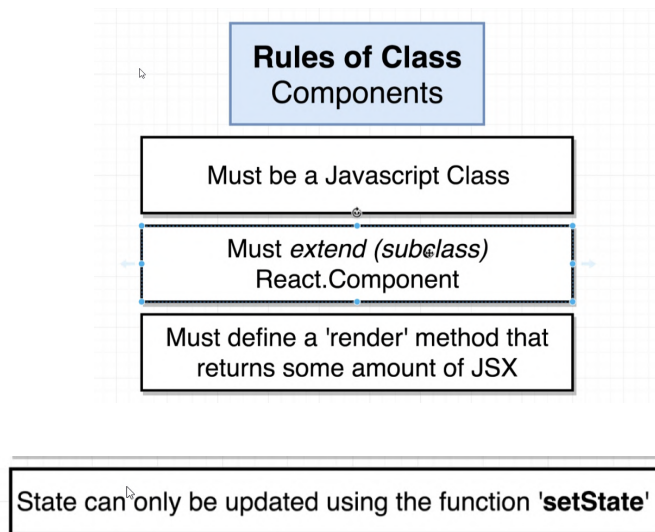
React is essentially a library for creating components, and each component has its own lifecycle which you can tap into and perform certain actions.

The five stages of the React component lifecycle are: **initialising**, **mounting**, **updating**, **unmounting**, and **error handling**.



✓ Class Component

Class components are JavaScript classes that extend the base **React.Component** class. It needs to include a render method, which returns HTML.

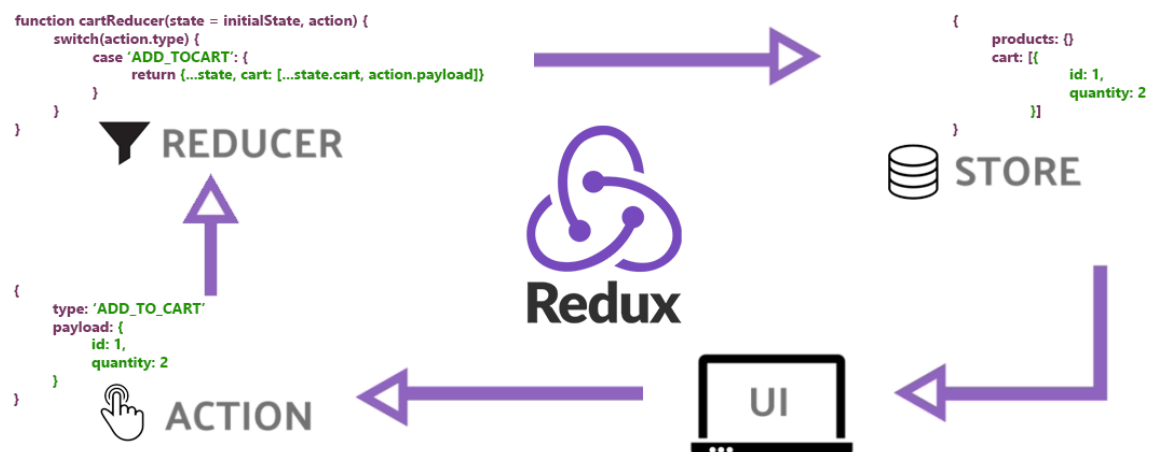


✓ Redux

Redux is a **popular state management library** for JavaScript. Redux allows developers to manage the state of their applications in a predictable and centralised way.

Redux involves creating a **reducer**, which is a pure function that takes in the current state and an action and returns a new state.

It also involves **creating action creators**, which are functions that dispatch actions that can update the data in the reducer.



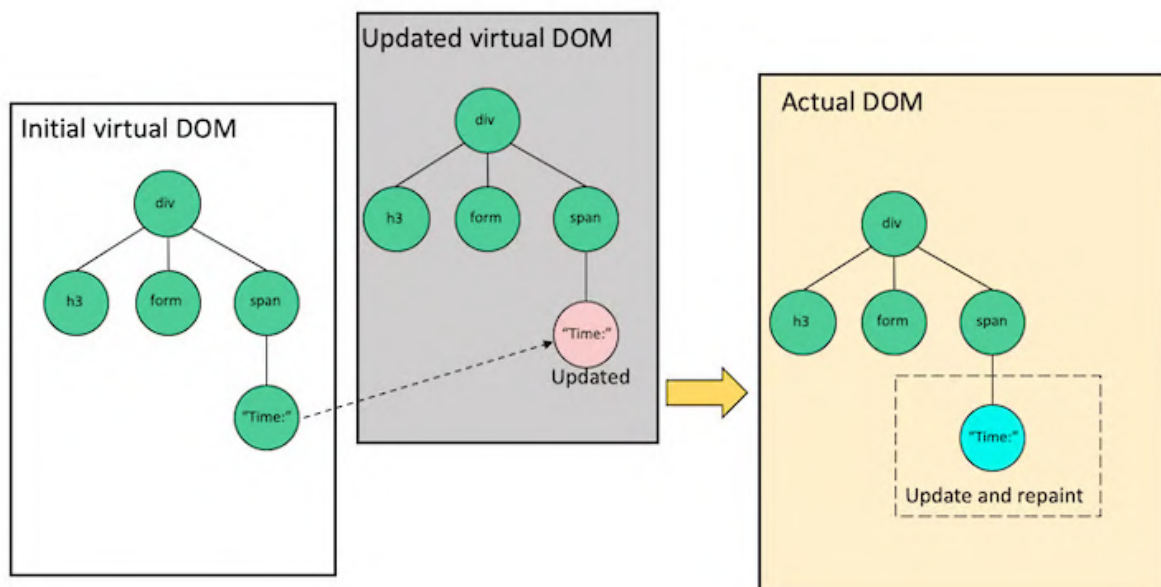
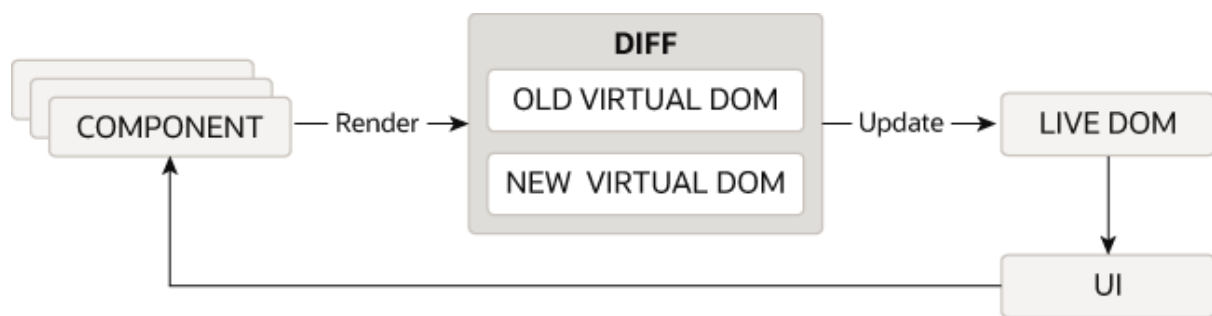
✓ Virtual DOM

Virtual DOM in React is an in-memory representation of the actual DOM.

It is a lightweight copy of the actual DOM that is used by React to determine which changes need to be made in the real DOM with the help of a **render() method**.

it requires less time to update the **Virtual DOM** compared to the **actual DOM**.

Whenever a **state changes**, the entire UI tree is re-rendered instead of just the single element that was changed. This process helps to improve the performance of web applications using React.



✓ Real vs Virtual DOM

Real DOM	Virtual DOM
1. It updates slow.	1. It updates faster.
2. Can directly update HTML.	2. Can't directly update HTML.
3. Creates a new DOM if element updates.	3. Updates the JSX if element updates.
4. DOM manipulation is very expensive.	4. DOM manipulation is very easy.
5. Too much of memory wastage.	5. No memory wastage.

Real DOM vs Virtual DOM

✓ What is JSX and how browser understands it?

- ✓ JSX is not a regular JavaScript object
- ✓ Browsers can read JavaScript objects only
- ✓ JSX file is converted to JS object by JSX transformers like Babel, before reaching browser

```
import React from 'react'

export class Hello extends React.Component {
  render() {
    return (
      <h1>Hello, {this.props.name}</h1>
    )
  }
}
```

An element

How JSX works

```
<MyButton color="blue" shadowSize={2}>
  Click Me
</MyButton>
```

Compiles to:

```
React.createElement(
  MyButton,
  {color: 'blue', shadowSize: 2},
  'Click Me'
)
```

✓ ES5 vs ES6 of React Syntax

ES5	ES6
<pre>var MyComponent = React.createClass({ display : function () {...}, render : function () { return { <div> <h1>Hello</h1> </div> }; } });</pre>	<pre>class MyComponent extends React.Component { show() {...} render() { return { <div> <h1>Hello</h1> </div> }; } }</pre>

✓ Error Handling in React

Error boundaries are React components that catch JavaScript errors anywhere in their child component tree, log those errors, and display a fallback UI instead of the component tree that crashed.

You can create a class component by extending React's Component class and defining a new **componentDidCatch(error, info)** lifecycle method. This method is used to catch JavaScript errors inside the component tree.

```
const ErrorBoundary = (WrappedComponent) => {
  return class ErrorBoundary extends React.Component {
    constructor(props) {
      super(props)
      this.state = { error: null, info: null }
    }
    componentDidCatch(error, info) {
      this.setState({ error: error, info: info })
    }
    render() {
      if (this.state.error) {
        //error boundary rendered
      }
      return <WrappedComponent {...this.props} />;
    }
  }
}
```

✓ How To Call Child Function From Parent Component In React

Take a look at this interactive example:

```
const Parent = () => {  
  const childFunc = React.useRef(null)  
  
  return (  
    <Child childFunc={childFunc} />  
    <button onClick={() => childFunc.current()}>Click me</button>  
  )  
}  
  
const Child = ({ childFunc }) => {  
  React.useEffect(() => {  
    childFunc.current = alertUser  
  }, [])  
  
  function alertUser() {  
    alert('You clicked!')  
  }  
  
  return null  
}  
  
render(<Parent />)
```

Live

✓ Fragment in React

- ✓ Fragments let us group a list of children without adding extra nodes to the DOM
- ✓ While rendering multiple elements will require a 'div' tag around the content as the render method will only render a single root node inside it at a time
- ✓ In React 16.2 version, Fragments were introduced, and we use them instead of the extraneous 'div' tag

```
render() {  
  return (  
    <React.Fragment>  
      <ChildA />  
      <ChildB />  
      <ChildC />  
    </React.Fragment>  
  )  
}
```


✓ Stateful & Stateless Component

Stateful components, also known as containers or smart components, are components that manage their own state and/or access the state of the application.

Stateless components, also known as presentational components, are components that do not maintain their own state and do not access the state of the application.

They are considered "dumb" since they are not responsible for making decisions or changes to the application state.

Stateful Component	Stateless Component
1. Stores info about components state change in memory	1. Calculates the internal state of the components
2. Have authority to change state	2. Do not have the authority to change state
3. Contains the knowledge of past, current and possible future changes in state	3. Contains no knowledge of past, current and possible future state changes
4. Stateless components notifies them about the requirement of the state change, then they send down the props to them.	4. They receive the props from the Stateful components and treat them as callback functions.

✓ HOC (higher-order component)

A higher-order component (**HOC**) is an advanced technique in React for reusing **component logic**. HOCs are not part of the React API, per se. They are a pattern that emerges from React's compositional nature. Concretely, **a higher-order component is a function that takes a component and returns a new component**.

```
const Wrapped = () => <h1>Wrapped Comp</h1>

const wrapper = (Comp) => () => {
  return <div><Comp /></div>
}
const FinalComponent = wrapper(Wrapped)

const App = () => {
  return (
    <div>
      <FinalComponent />
    </div>
  )
}
```


✓ Context API

Context provides a way to pass data through the component tree without having to pass props down manually at every level.

Context is designed to share data that can be considered “**global**” for a tree of React components, such as the current authenticated user

1. Create a context object:

```
const MyContext = React.createContext();
```

Copy code

2. Provide the context value in a parent component:

```
class ParentComponent extends React.Component {
  state = {
    name: 'John Doe',
  };

  render() {
    return (
      <MyContext.Provider value={this.state.name}>
        /* Child components go here */
      </MyContext.Provider>
    );
  }
}
```

Copy code

3. Consume the context value in a child component:

```
class ChildComponent extends React.Component {
  static contextType = MyContext;

  render() {
    return (
      <div>
        Hello, {this.context}
      </div>
    );
  }
}
```

Copy code

✓ useEffect Hook as a LifeCycle Hook

React Hooks are a new addition in **React 16.8**. They provide a way to use state and lifecycle features in **functional components** as an alternative to classes.

React Hooks allow you to use features like **componentDidMount**, **componentDidUpdate**, and **componentWillUnmount** without having to use a class component.

You can also access state and other React features without writing a class.

[How the useEffect Hook Works \(with Examples\)](#)

```
useEffect(() => {
  console.log('I will run on every render');
});

useEffect(() => {
  console.log('I will run only on first render');
}, []);

useEffect(() => {
  console.log('I will run evrytym x changes');
}, [x]);

useEffect(() => {
  // Some task

  return () => {
    console.log('I do cleanups');
    console.log('will first run on component mount then,
    will run before useeffect and lastly before
    unmounting')
  };
});
```

✓ Implement Debounce

Debouncing in React is a technique that can be used to make sure expensive operations, such as API requests, don't run too often.

Instead of running the operations on **every keystroke**, it can be debounced so that it runs only after a certain amount of time has elapsed since the last time it ran.

This helps to ensure that resources are not wasted by running too many operations at once

Here in example we're creating a useDebounce Hook to achieve it.

```
// Use Debounce to improve performance
const useDebounce = (func, milliseconds) => {
  const time = milliseconds || 400;
  let timer;

  return event => {
    if (timer) {
      clearTimeout(timer);
    }

    timer = setTimeout(func, time, event);
  };
};

const resize = useDebounce(() => { ... });

window.addEventListener('resize', resize);
```

✓ Controlled and uncontrolled components

- A **controlled component** is a component where React is in control and is the single source of truth for the form data.
- An **uncontrolled component** is where your form data is handled by the DOM, instead of inside your React component.

Here's an example of a controlled input component:

In this example, the parent component **ParentComponent** maintains the state of the input value, and passes it as a prop to the controlled input component **ControlledInput**.

The **handleChange** callback function is also passed as a prop to the **ControlledInput** component and is invoked when the input value changes.

```
class ControlledInput extends React.Component {
  render() {
    return (
      <input
        value={this.props.value}
        onChange={this.props.onChange}
      />
    );
  }
}

class ParentComponent extends React.Component {
  state = {
    inputValue: '',
  };

  handleChange = (event) => {
    this.setState({ inputValue: event.target.value });
  };

  render() {
    return (
      <ControlledInput
        value={this.state.inputValue}
        onChange={this.handleChange}
      />
    );
  }
}
```

Here's an example of an **uncontrolled** input component:

```
class UncontrolledInput extends React.Component {
  inputRef = React.createRef();

  handleSubmit = (event) => {
    event.preventDefault();
    console.log(this.inputRef.current.value);
  };

  render() {
    return (
      <form onSubmit={this.handleSubmit}>
        <input ref={this.inputRef} />
        <button type="submit">Submit</button>
      </form>
    );
  }
}
```

✓ UseMemo Vs UseCallback

useCallback()

useCallback will remember a **function** between renders.

Used to fix performance issues, e.g. when **changing** a function can cause expensive components to re-render.

```
const SomeComponent = () => {
  // Whenever SomeComponent re-renders, onItemClick fn reference
  // stays the same, and ExpensiveComponent won't re-render
  const onItemClick = React.useCallback(() => {
    /* do something */
  }, [])
  <ExpensiveComponent onItemClick={onItemClick} />
};

const ExpensiveComponent = React.memo(({onItemClick}) => {
  /* ... */
})
```

useMemo()

useMemo will remember a **value** between renders.

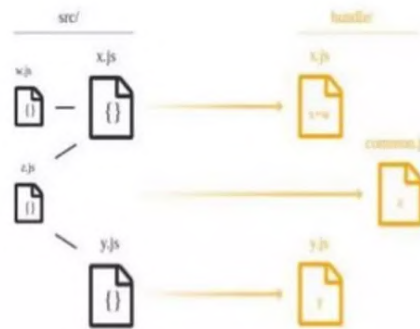
Used to avoid re-calculating expensive operations.

```
// expensive function that takes some time to calculate
import searchUsersByOccupation from 'utils/users'

const UsersList = ({ users, occupation }) => {
  // ⚡ Hey React, remember this value between renders
  // Only re-calculate when user's list or occupation changes
  const usersByOccupation = React.useMemo(() => {
    return searchUsersByOccupation(users, occupation)
  }, [users, occupation]);
};
```

✓ Code Splitting

- It allows us to split our code into various chunks which you can then load on demand.
- It helps us to reduce the initial response size required to render the page,



Example of code splitting

```
const Home = Loadable({
  loader: () => import('./Home'),
  loading: () => <div>Loading ... </div>,
})

const Blog = Loadable({
  loader: () => import('./Blog'),
  loading: () => <div>Loading ... </div>,
})

const App = () => (
  <Router>
    <Switch>
      <Route exact path="/" component={Home} />
      <Route path="/blog" component={Blog} />
    </Switch>
  </Router>
);
```

✓ React is One way or two way Data Binding ?

React has one way data binding, which means, data flow is from owner to child only, the child can't update the data directly. It will need to dispatch some actions/call the function required to update the data.

✓ Pure component

A React component is considered pure if it renders the same output for the same state and props. For this type of class component, React provides the **PureComponent** base class. Class components that extend the **React.PureComponent** class are treated as pure components.

Example of Pure component 👍

```
class PercentageStat extends React.PureComponent {

  render() {
    const { label, score = 0, total = Math.max(1, score) } = this.props;

    return (
      <div>
        <h6>{ label }</h6>
        <span>{ Math.round(score / total * 100) }%</span>
      </div>
    )
  }
}

export default PercentageStat;
```


✓ What is the difference between state and props?

Both props and state are plain JavaScript objects. While both of them hold information that influences the output of render, they are different in their functionality with respect to components. i.e

- **Props** get passed to the component similar to function parameters
- **State** is managed within the component similar to variables declared within a function

```
class PetsList extends React.Component {  
  constructor(props) {  
    super(props);  
    this.state = {  
      pets: pets  
    };  
  }  
  render () {  
    return (  
      <div>  
        <h3>Pets List: </h3>  
        {  
          this.state.pets.map((v,idx) => {  
            return <SinglePet key={idx}  
              name={v.name}  
              image={v.image}  
              desc={v.desc}>  
          }  
        )  
      </div>  
    )  
  }  
}
```

```
class SinglePet extends React.Component {  
  render () {  
    return (  
      <div>  
        <h3>{this.props.name}</h3>  
        <img src={this.props.image} alt="" />  
        <p>{this.props.desc}</p>  
      </div>  
    )  
  }  
}
```

props

Coding 1 - Stopwatch Timer

```
import './styles.css';
import { useRef, useState } from 'react';
export const App = () => {
  const stopWatchRef = useRef(null);
  const [value, setValue] = useState(0);

  const startCounter = () => {
    if (!stopWatchRef.current) {
      console.log("passed");
      stopWatchRef.current = setInterval(() => {
        setValue((preVal) => preVal + 1);
      }, 1000);
    }
  };

  const stopCounter = () => {
    clearInterval(stopWatchRef.current);
    stopWatchRef.current = null;
  };

  return (
    <div>
      <h1> Stopwatch Counter </h1>
      {value}
      <hr />
      <button onClick={startCounter}> Start </button> &nbsp;
      <button onClick={stopCounter}> Stop </button>
    </div>
  );
};

export default App;
```

Coding 2 - Simple Pagination

```
export default function App() {
  const items = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15];
  const itemsPerPage = 5;
  return (
    <div className="App">
      <h1> Pagination </h1>
      <Pagination items={items} itemsPerPage={itemsPerPage} />
    </div>
  );
}

const Pagination = ({ items, itemsPerPage }) => {
  const totalPages = Math.ceil(items.length / itemsPerPage);
  const pages = Array.from(Array(totalPages)
    .keys()).map((i) => i + 1);
  return (
    <div>
      {pages.map((page) => (
        <button key={page}>{page}</button>
      ))}
    </div>
  );
};
```

Output :

Pagination

1	2	3
---	---	---

Coding 3 - Fetch Api (Custom Hook)

```
function App() {
  const API = 'https://jsonplaceholder.typicode.com/posts'
  const { data: posts, error, isLoading } = useFetch(API)
  return (
    <> <h1>Posts</h1>
      {isLoading && <h1>Loading...</h1>}
      {error && <h1>{error}</h1>}
      {posts.map(post => <div key={post.id}>
        <h3> {post.title}</h3>
        <p> {post.body}</p>
        <hr />
      </div> )}
    </>)
}
```

```
// useFetch Custom Hook to fetch the API
function useFetch(endpoint, resType = []) {
  const [data, setData] = useState(resType)
  const [error, setError] = useState('')
  const [isLoading, setIsLoading] = useState(true)
  useEffect(() => {
    async function fetchData() {
      setIsLoading(true)
      try {
        const res = await axios.get(endpoint)
        setData(res.data) setIsLoading(false)
      } catch (e) {
        setError(e.message) setIsLoading(false)
      }
    }
    fetchData()
  }, [endpoint])
  return { data, isLoading, error }
}
```

Live Interview - MNC companies Interview videos

 Fresher's Frontend | Javascript 🎉 | ReactJs Interview (Mock) [Most Asked Questions ...

 HCL Technologies Frontend + ReactJs interview | 🎉 Selected | JavaScript Interview | ...

 Cognizant ReactJs Interview (Round-1) | 🎉 Selected | ReactJs & Javascript | Frontend

 Mindtree React Js and Javascript interview | 🎉 Selected | ReactJs Interview questio...

 Infosys ReactJs coding Interview | 🎉 Selected | JavaScript Coding Interview | Experi...

Projects

 Build and Deploy 5 JavaScript & React API Projects in 10 Hours - Full Course | RapidAPI

Important Links

- [React Beta Docs](#)
- [Ace the Javascript Interview – Manu Arora](#)
- [React lifecycle methods diagram](#)
- [Refs and the DOM – React](#)
- [React Tutorial: Learn React JS - Free 11-Hour Course](#)
- [A 2-minute React app](#)
- [React v18.0 – React Blog](#)

Social Media



Join Our Telegram Group

- https://t.me/Reactjs_Interview

- https://t.me/React_JS_India



Dev. Aditya - YouTube (10k)

Copyright © 2023 [DevAditya](#). All Rights Reserved.

This e-file is property of DevAditya. No part of this e-file may be reproduced or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the author, except in the case of brief quotations embodied in critical reviews and certain other noncommercial uses permitted by copyright law. For permission requests, contact the author.